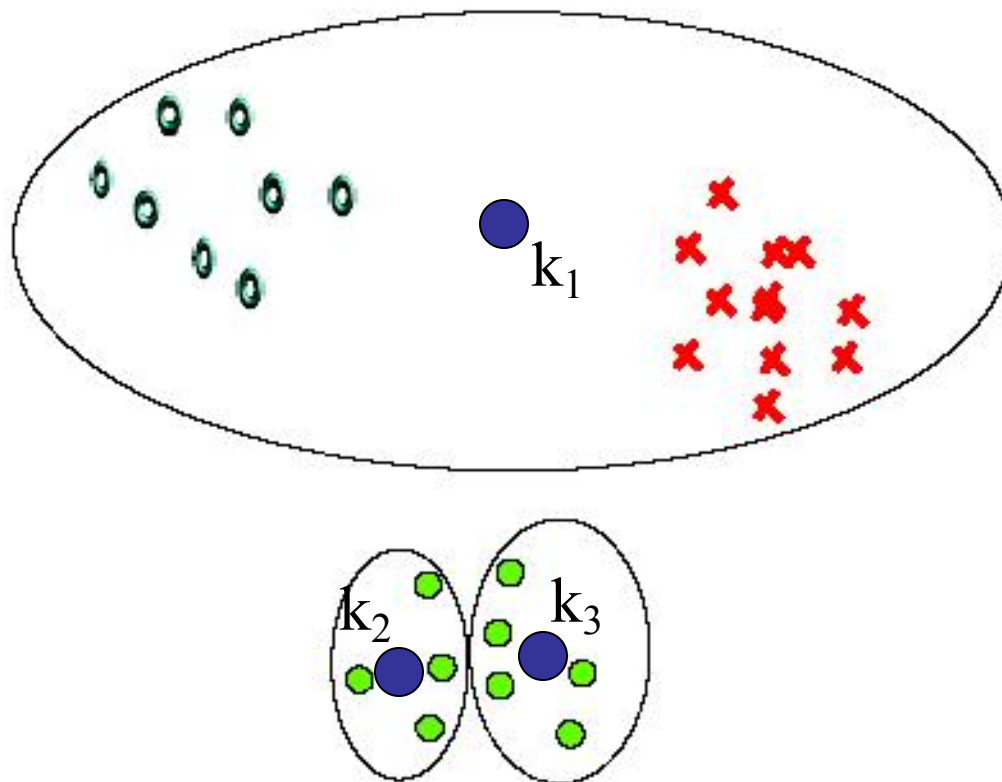


基于密度的聚类方法

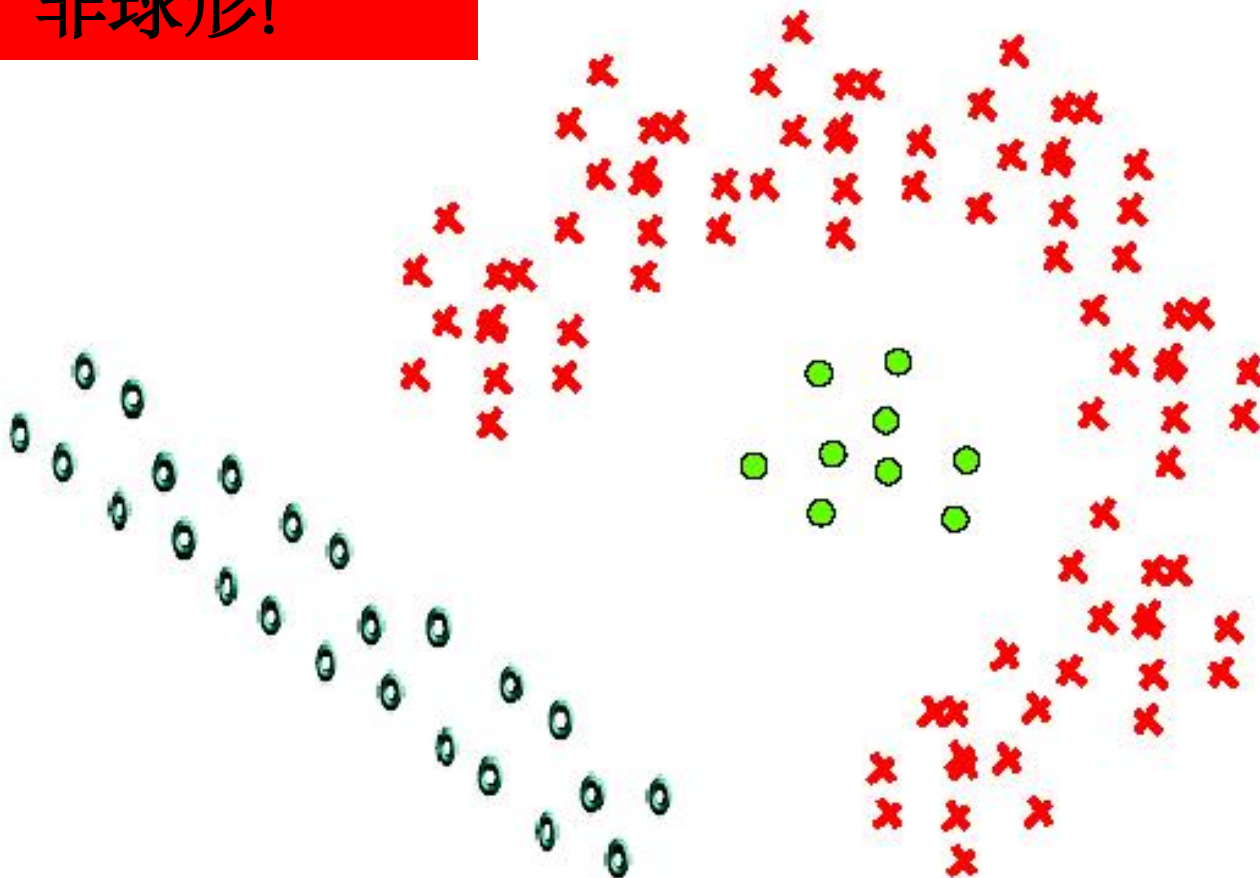
- 之前介绍的基于划分的聚类方法（如K-means）是基于数据对象间的距离实现的
- 该类方法具有三个先天的缺陷
 - 必须事先输入一个参数k
 - 聚类的结果与初始中心点的选择有很大关系，如果初始中心点选择不好的话，则聚类结果很差
 - 只能适用于球形的族

不好的初始中心点导致聚类结果很差



无法找出非球形的族

非球形!



基于密度的方法

主要思想是：只要临近区域的密度（对象或数据点的数目）超过某个阈值，就继续聚类。

也就是说，对给定类中的每个数据点，在一个给定范围的区域中必须包含至少某个数目的点。

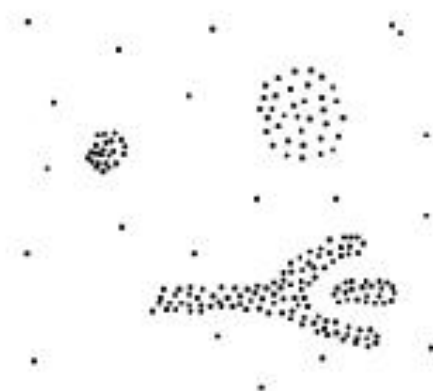
这样的方法可以用来过滤“噪音”数据，发现任意形状的簇。



database 1



database 2



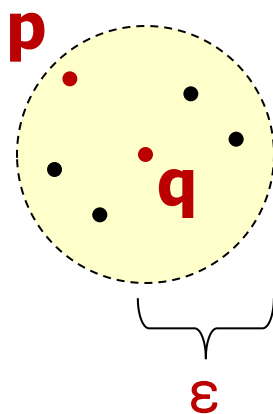
database 3

DBSCAN

- Density Based Spatial Clustering of Applications with Noise
- 基本思想:
 - 如果一个数据对象p和另一个数据对象q是密度相连 (**density connected**) 的, 则p和q属于同一个族
 - 如果某个数据对象到任何一个其它数据对象都不密度相连, 则认为该数据对象是一个噪音

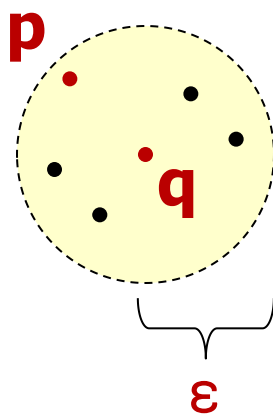
定义1: ε -邻域

- 以二维空间为例，给定一个数据对象 q 和一个参数 ε ，则以 q 为圆心，以 ε 为半径可以画圆，该圆所在的区域称为数据对象 q 的 ε -邻域
- 下图所示的虚线内的黄色区域为数据对象 q 的 ε -邻域：



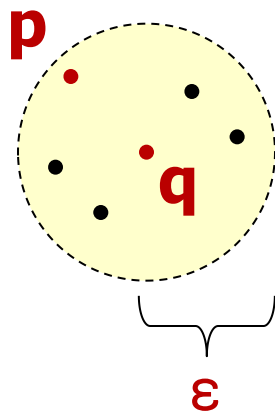
定义2: 核心对象

- 给定一个参数MinPts, 如果数据对象 q 的 ϵ -邻域内至少包含MinPts个数据对象, 则称 q 为**核心对象**
- 下图中, 如果设定MinPts小于或等于6的话, 则 q 是一个核心对象, 否则 q 不是一个核心对象。



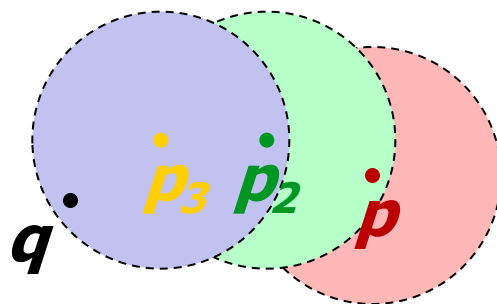
定义3: 直接密度可达

- 给定一个数据对象集合D, 如果q是一个核心对象, 且p在q的 ε -邻域内, 则说p从q直接密度可达(directly density-reachable)
- 如下图, 如果q是一个核心对象, 由于p在q的 ε -邻域中, 则p从q直接密度可达



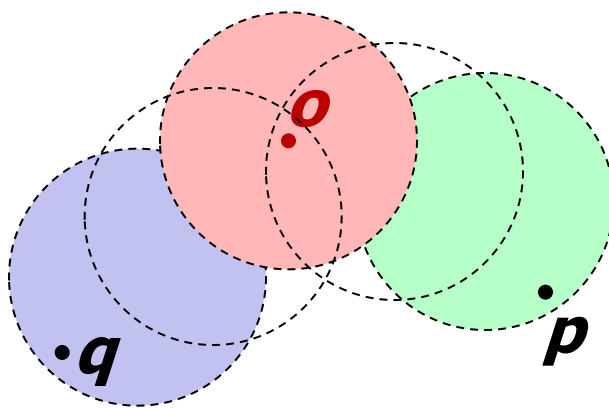
定义4: 密度可达

- 给定一个数据对象集合 D , 两个参数 ϵ 和 MinPts , 如果存在一系列数据对象: $p_1, p_2, \dots, p_n, p_1=p, p_n=q$, 并且对任一数据对象 $p_i \in D, 0 < i < n$, p_{i+1} 从 p_i 直接密度可达, 则称 p 到 q 密度可达(density-reachable)
- 密度可达具有传递性, 但不具有对称性
- 如下图, p 到 p_2 直接密度可达, p_2 到 p_3 直接密度可达, p_3 到 q 直接密度可达, 所以 p 到 q 密度可达。但反过来, q 到 p 不一定密度可达 (为什么?)



定义5: 密度相连

- 给定一个数据对象集合D，两个参数 ϵ 和MinPts，如果对象集合D中存在一个数据对象o，并且o到数据对象p和q密度可达，则数据对象p和q密度相连（density-connected）。
- 和密度可达的概念相反，密度相连的概念具有对称性，但不具有传递性
- 如下图，o到p和q同时密度可达，则p和q之间密度相连。反过来，q和p之间也密度相连。



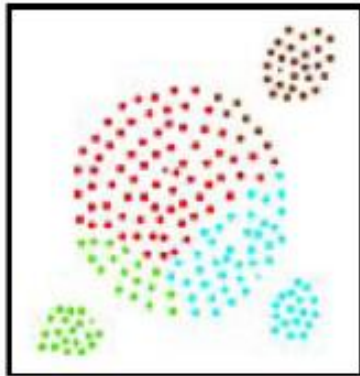
基于密度的聚类算法

根据上述定义，在基于密度的聚类方法中，一个族被定义为所有密度相连的数据对象的最大集合

DBSCAN工作过程

- 从任意一个数据对象 p 开始
 - 如果 p 是一个核心对象，则根据输入的两个参数 ϵ 和MinPts，通过广度优先搜索提取所有从 p 密度可达的数据对象，将它们标记为当前族，并从它们进一步扩展。
 - 如果 p 是一个边界对象，则将 p 标记为噪声，再随机选取另外一个数据对象进行处理。
 - 依次进行下去，直到找到一个完整的族。
- 然后再选择一个新的其它数据对象开始扩展，得到下一个族，算法一直进行到所有的数据对象都被标记过为止

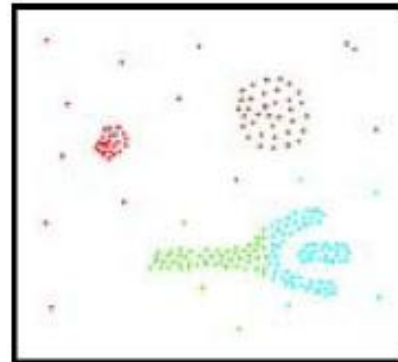
性能比较



database 1

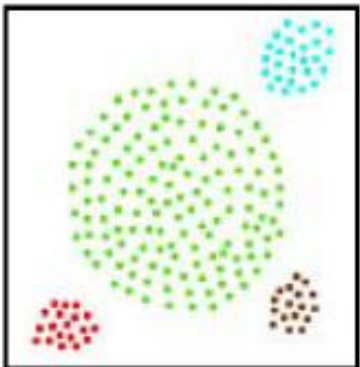


database 2



database 3

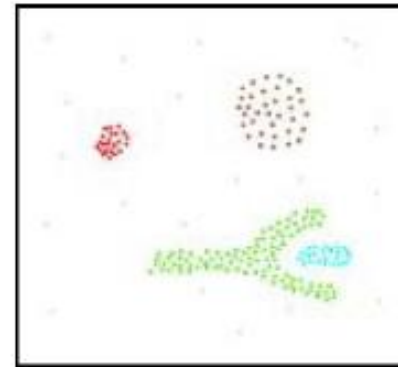
Clarans



database 1



database 2



database 3

DBSCAN

DBSCAN密度聚类例子

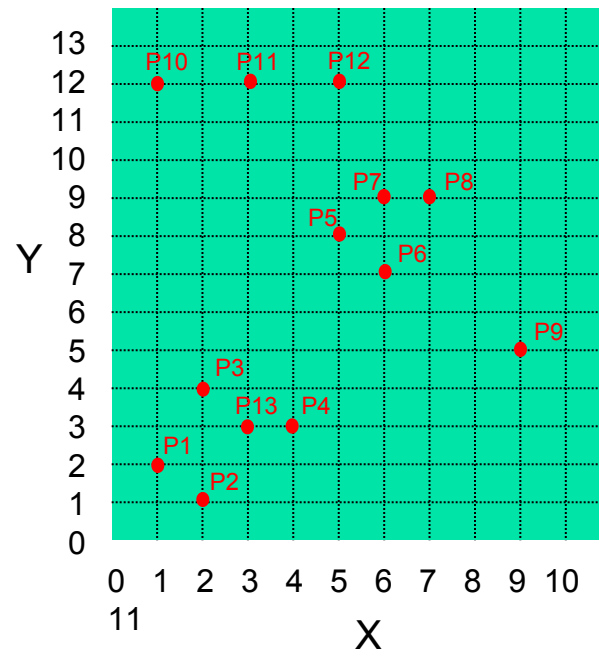
1. 将所有对象标记为核心点、边界点或噪声点
2. 如果选择的点是核心点，则找出所有从该点出发的密度可达对象形成簇
3. 如果该点是非核心点，将其指派到一个与之关联的核心点的簇中
4. 重复以上步骤，直到所点都被处理过

举例：有如下13个样本点，使用DBSCAN进行聚类

	P1	P2	P3	P4	P5	P6	P7	P8	P9	P10	P11	P12	P13
X	1	2	2	4	5	6	6	7	9	1	3	5	3
Y	2	1	4	3	8	7	9	9	5	12	12	12	3

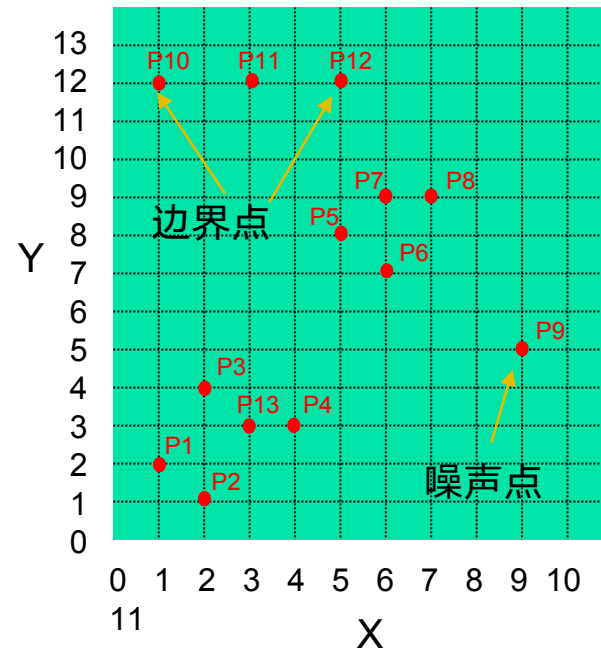
DBSCAN密度聚类例子

- 对每个点计算其邻域 $Eps=3$ 内的点的集合
- 集合内点的个数超过 $MinPts=3$ 的点为核心点



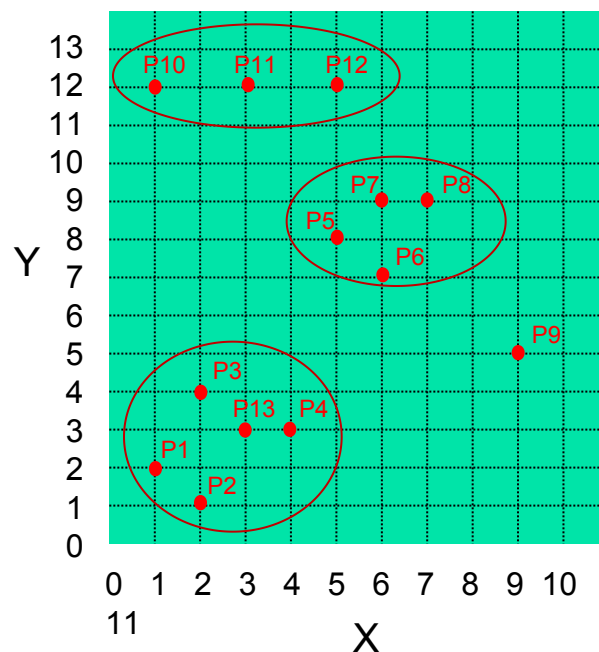
DBSCAN密度聚类例子

- 查看剩余点是否在核心点的邻域内，若在，则为边界点，否则为噪声点

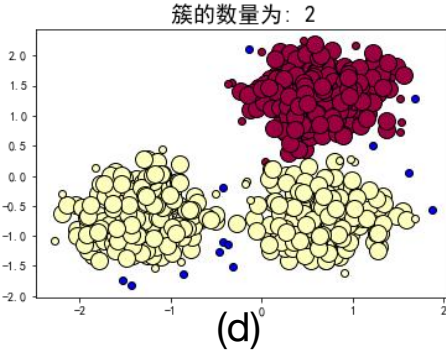
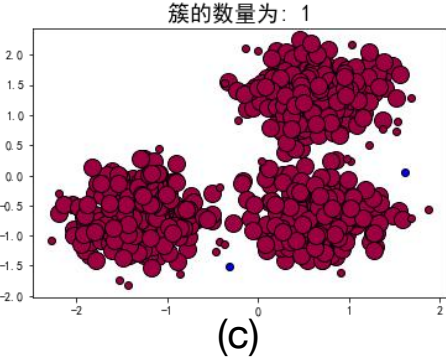
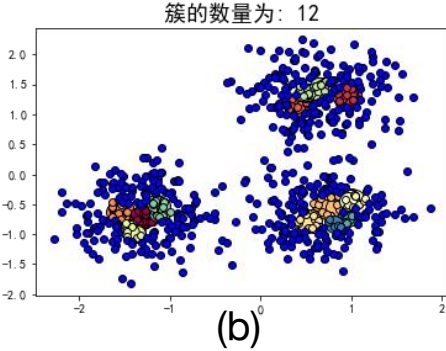
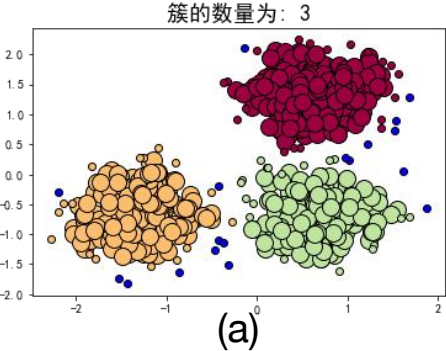


DBSCAN密度聚类例子

- 将距离不超过 $Eps=3$ 的点相互连接，构成一个簇，核心点邻域内的点也会被加入到这个簇中



DBSCAN的超参数



DBSCAN超参数案例

评价指标	超参数	(a)	(b)	(c)	(d)
		eps=0.3 minPts=10	eps=0.1 minPts=10	eps=0.4 minPts=10	eps=0.3 minPts=6
估计的簇的数量		3	12	1	2
估计的噪声点		18	516	2	13

这个案例中，当：
eps=0.3， minPts=10的时候，
DBSCAN达到最优效果。

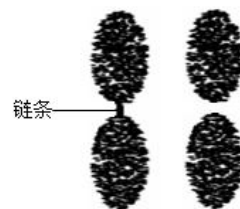
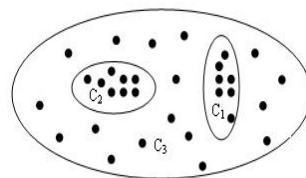
DBSCAN总结

优点

- 不需要事先确定族的个数
- 聚类速度快。使用索引（例如R*-tree）时，DBSCAN的时间复杂度为 $O(n \log n)$ ， n 为数据库中数据对象的个数，否则，DBSCAN的时间复杂度为 $O(n^2)$
- 对噪声数据不敏感
- 能发现任意形状的族

缺点

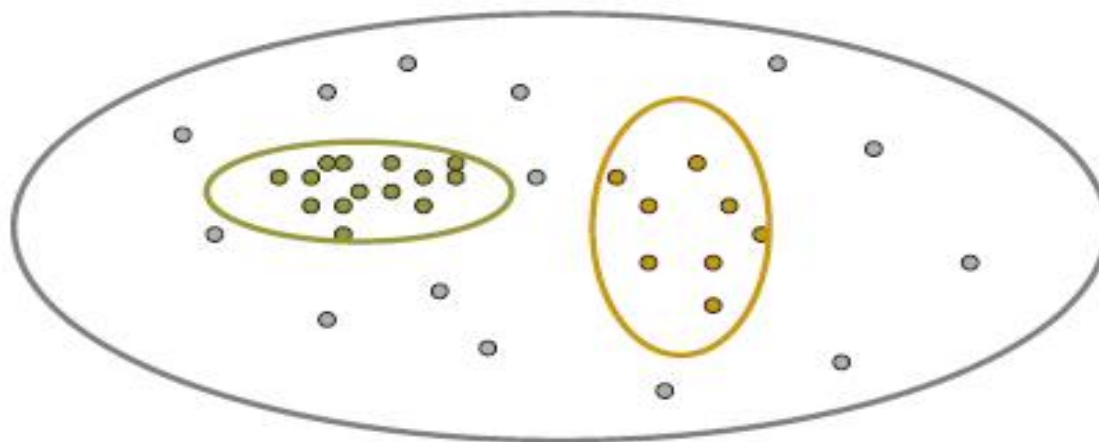
- 输入参数 ϵ 和MinPts的值较难确定
- 当数据库中数据对象的密度分布不均匀时，用相同的参数值可能得不到好的聚类结果



- 可能会产生“链条”现象
- 使用R*-tree索引时，由于R*-tree在高维空间中不够有效，导致DBSCAN算法在处理高维数据时性能下降

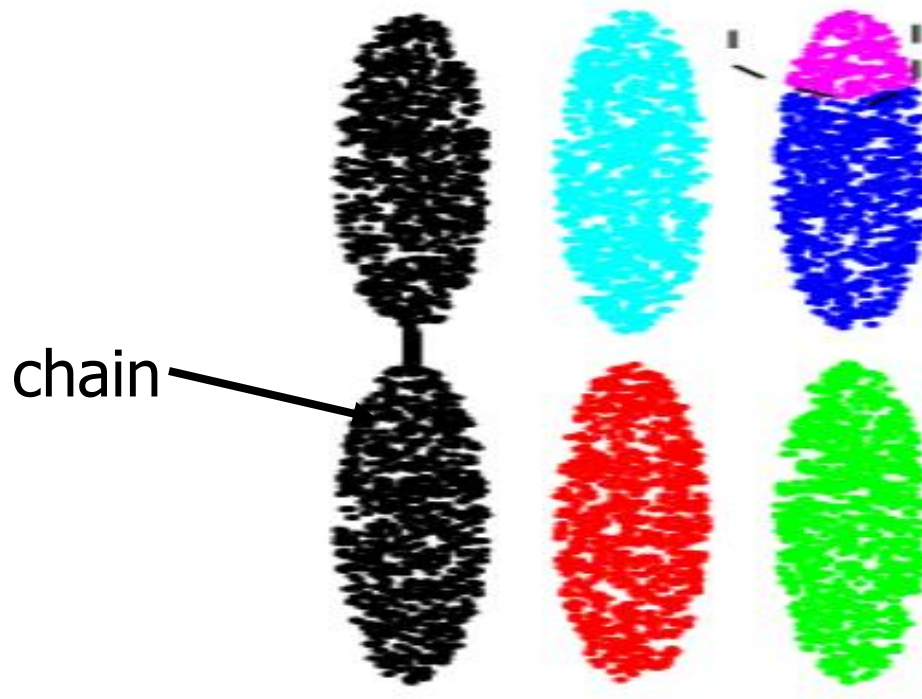
[Problems of DBSCAN]

- Different clusters may have very different densities.
- Clusters may be in hierarchies.



OPTICS: 针对该问题进行改进

The Chaining Problem

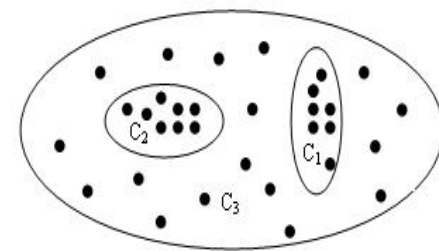


CURE : 针对该问题进行改进

OPTICS算法

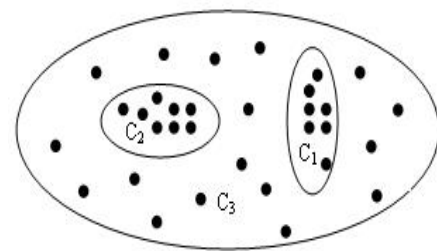
- 核心思想：为每个数据对象计算出一个顺序值（**ordering**）。这些值代表了数据对象的基于密度的族结构，位于同一个族的数据对象具有相近的顺序值。根据这些顺序值将全体数据对象用一个图示的方式排列出来，根据排列的结果就可以得到不同层次的族。

OPTICS算法



- 考察DBSCAN，可以发现，对于一个恒定的MinPts值来说， ϵ 取值较小时得到的聚类结果完全包含在根据较大的 ϵ 取值所获得的聚类结果中。
- 如图，当 ϵ 取值较小时，得到的聚类结果是C1和C2，当 ϵ 取值较大时，得到的聚类结果是C3。
- 可以看到，C1和C2是包含在C3中的。换句话说，C1、C2、C3间具有层次关系，C3可以看作是C1和C2的父亲，而C1和C2可以看作是C3的孩子。
- 因此，在生成族的时候，最好能够将位于不同层次上的族同时构建出来，而不是根据某个特定的 ϵ 值仅仅构建其中的一层。

OPTICS算法



- 为了同时构建不同层次上的族，数据对象应当以特定的顺序来处理。这个顺序称为族序 (**cluster-ordering**)，它决定了对象扩展时的次序。
- 为了使较低层次上的族（这些族的数据密度较大）能够首先构建完成，在进行对象扩展时，应该优先选择那些根据最小的 ϵ 取值而密度可达的对象。
- 基于这个思想，每个数据对象需要存储两个值，一个是核心距离 (**core-distance**)，另一个是可达距离 (**reach-distance**)。

OPTICS算法-核心距离

- 给定一个数据对象集合D，两个参数 ϵ 和MinPts，一个对象O，如果O是一个核心对象，则O的**核心距离** (core-dist) 是使得O能成为核心对象的最小半径值(该值小于等于 ϵ)。如果O不是核心对象，则O的核心距离没有定义。即：

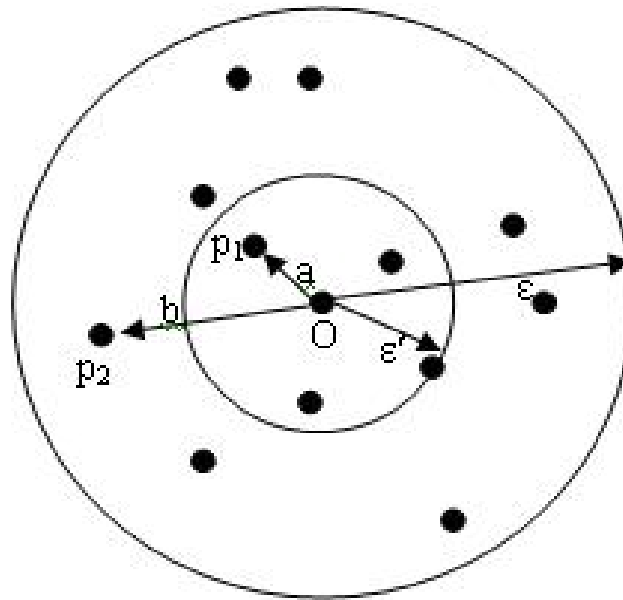
$$\text{core-dist}_{\epsilon, \text{MinPts}}(o) = \begin{cases} \infty & \text{if } |\text{rangeQuery}(o, \epsilon)| < \text{MinPts} \\ \text{MinPts} - \text{dist}(o) & \text{otherwise} \end{cases}$$

OPTICS算法-核心距离

- 其中 $|\text{rangeQuery}(O, \varepsilon)| < \text{MinPts}$ 表示在 O 的 ε -邻域的数据对象的个数小于 MinPts 个，说明在这种情况下 O 不是一个核心对象。
- 反之，当 O 是一个核心对象时， $\text{MinPts-dist}(O)$ 表示的就是使得 O 的 ε -邻域能够包含 MinPts 个数据对象的最小半径值。

OPTICS算法-核心距离

- 例如，给定 $\text{MinPts}=5$ ，则 ϵ' 表示的半径就是对象O的核心距离



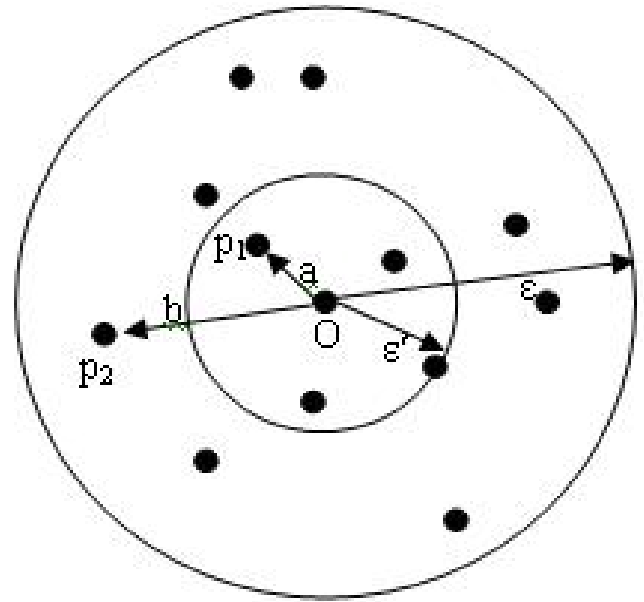
OPTICS算法-可达距离

- 给定一个数据对象集合D，两个参数 ϵ 和MinPts，一个对象O，如果O是一个核心对象，则O与另一个对象p间的可达距离 (reachability-distance) 是O的核心距离和O与p的欧几里得距离之间的较大值。如果O不是一个核心对象，O与p之间的可达距离没有定义。即：

$$reach-dist_{\epsilon, MinPts}(p, o) = \max(core-dist_{\epsilon, MinPts}(o), dist(p, o))$$

OPTICS算法-可达距离

- 由于 p_1 和 O 之间的距离 a 小于对象 O 的核心距离, 即 $\text{dist}(p_1, O) < \text{core-dist}(O)$, 所以 p_1 和 O 之间的可达距离就是对象 O 的核心距离。即 $\text{reach-dist}(p_1, O) = \text{core-dist}(O) = \epsilon'$ 。
- 由于 p_2 和 O 之间的欧几里得距离 b 大于对象 O 的核心距离, 即 $\text{dist}(p_2, O) > \text{core-dist}(O)$, 所以 p_2 和 O 之间的可达距离就是 p_2 和 O 之间的欧几里得距离。即 $\text{reach-dist}(p_2, O) = \text{dist}(p_2, O)$ 。



OPTICS算法的工作过程

- 第一个阶段计算每个对象的核心距离和可达距离，生成族序；
- 第二个阶段进行聚类，在聚类的过程中，只需要用到第一阶段所生成的对象之间的族序信息，不再需要其它的信息。

OPTICS算法：第一阶段

- 采用和DBSCAN算法类似的工作过程。同样从任意一个数据对象 p 开始，如果 p 是一个核心对象，则根据输入的两个参数 ϵ 和MinPts，提取所有从 p 直接密度可达的数据对象，计算它们的核心距离和可达距离，并将它们放入待处理队列 Q 中。

OPTICS算法：第一阶段

- 接下来，算法从 Q 中选取一个具有最小可达距离值的对象 q 进行进一步的扩展。同样，首先检查 q 是否是核心对象，如果是，则根据输入参数 ϵ 和 **MinPts**，提取所有从 q 直接密度可达的数据对象，计算它们的核心距离和可达距离，并将它们放入待处理队列 Q 中。如果 q 不是核心对象，则什么都不做。

OPTICS算法：第一阶段

- 需要注意的是，对 q 进行扩展时，还需要对队列 Q 中的数据对象的可达距离进行更新，保证其存储的是到最近的核心对象的距离。算法一直进行到所有的数据对象都被处理过为止。

OPTICS算法： 第二阶段

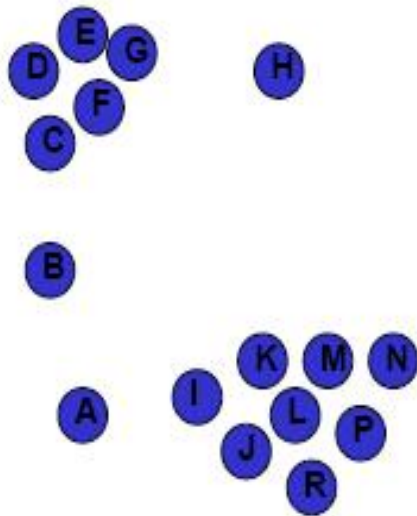
- 根据第一阶段所生成的族序和特定的 ϵ_i ($0 \leq \epsilon_i \leq \epsilon$) 值，生成相应的族。具体过程是：根据族序逐个处理每一个对象。

OPTICS算法：第二阶段

- 对任一对象 p ，首先看 p 的可达距离是否大于 ϵ_i 。如果是，则说明在 p 之前所处理过的对象，没有一个对象到 p 是可达的。（这是因为，如果某个对象到 p 可达的话， p 的可达距离不可能大于 ϵ_i ）。因此，如果 p 的可达距离大于 ϵ_i 的话，需要进一步考察 p 的核心距离。
 - 如果 p 的核心距离小于 ϵ_i ，则说明 p 是一个核心对象，这时创建一个新族
 - 否则，将 p 标记为噪声。
- 如果 p 的可达距离小于等于 ϵ_i 的话，则直接将 p 标记为当前族。

OPTICS Algorithm

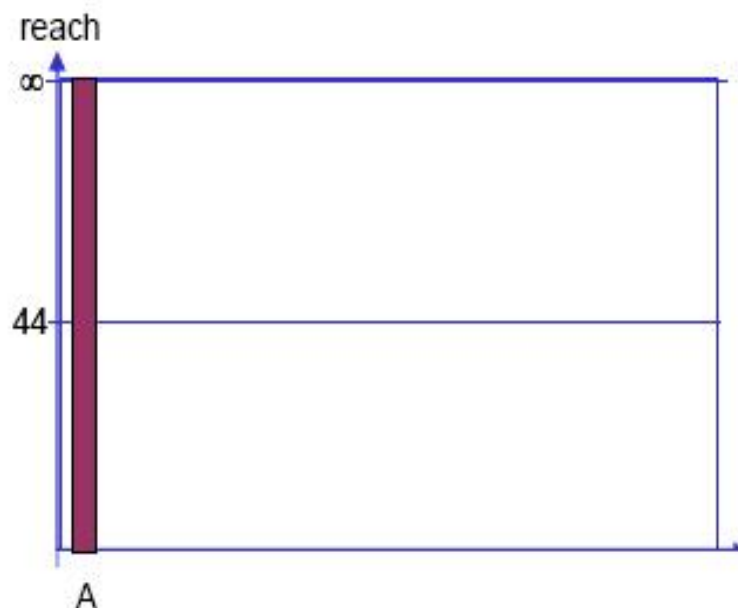
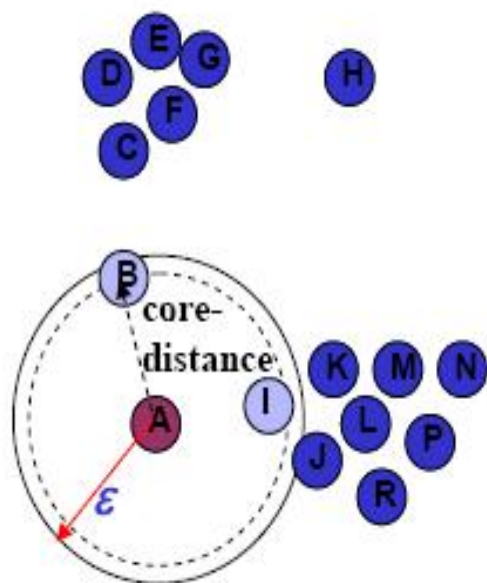
- Example Database (2-dimensional, 16 points)
- $\epsilon = 44$, $MinPts = 3$



seedlist:

OPTICS Algorithm

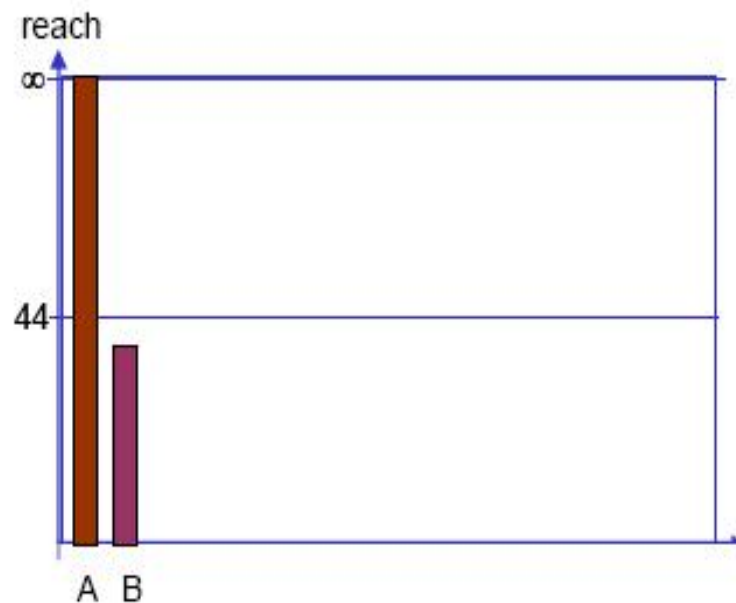
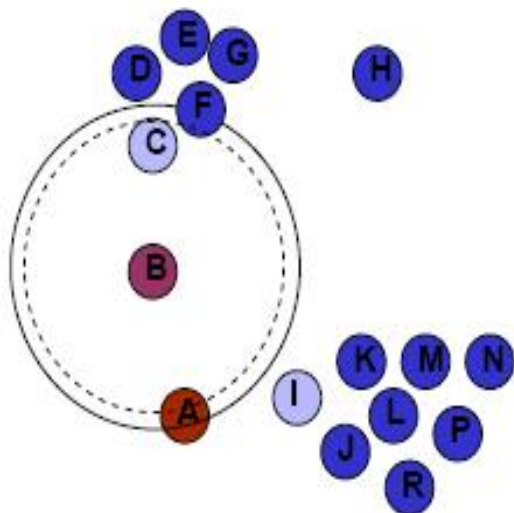
- Example Database (2-dimensional, 16 points)
- $\epsilon = 44$, $MinPts = 3$



seedlist: (B,40) (I, 40)

OPTICS Algorithm

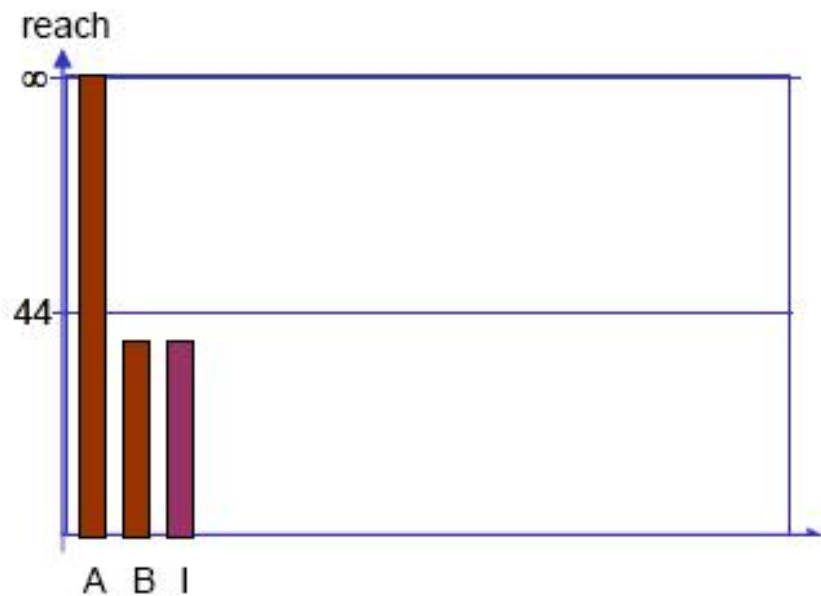
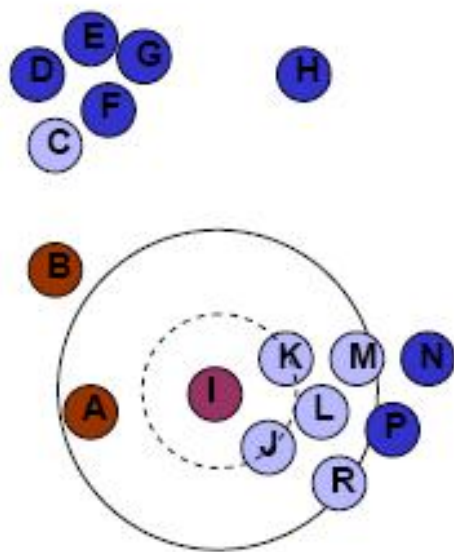
- Example Database (2-dimensional, 16 points)
- $\epsilon = 44$, $MinPts = 3$



seedlist: (I, 40) (C, 40)

OPTICS Algorithm

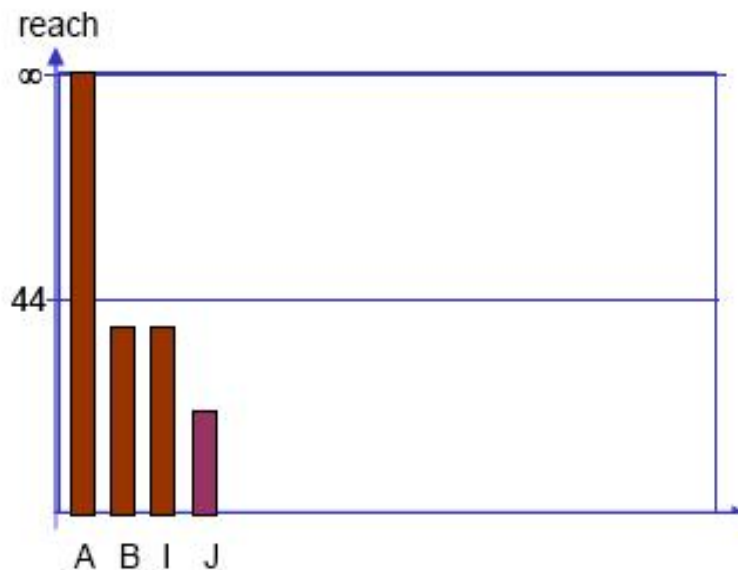
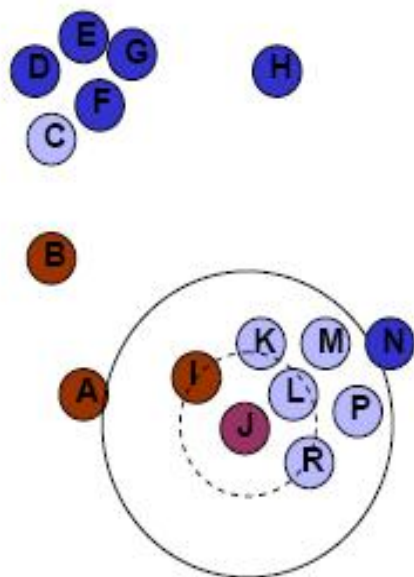
- Example Database (2-dimensional, 16 points)
- $\epsilon = 44$, $MinPts = 3$



seedlist: (J, 20) (K, 20) (L, 31) (C, 40) (M, 40) (R, 43)

OPTICS Algorithm

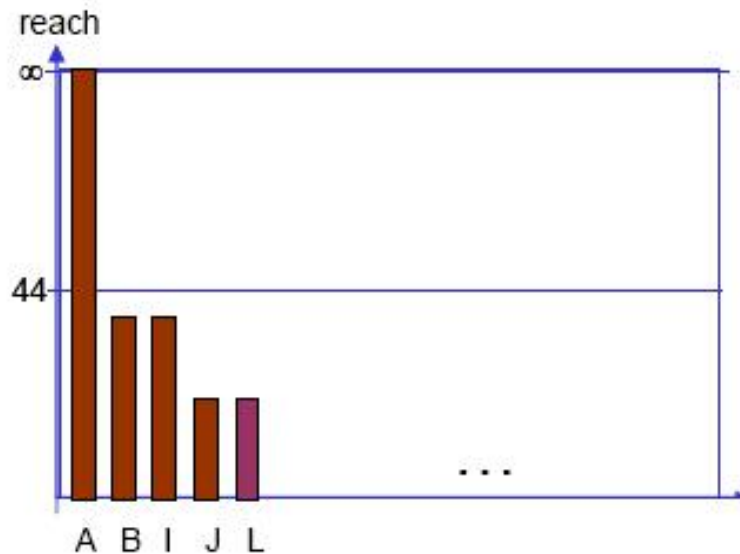
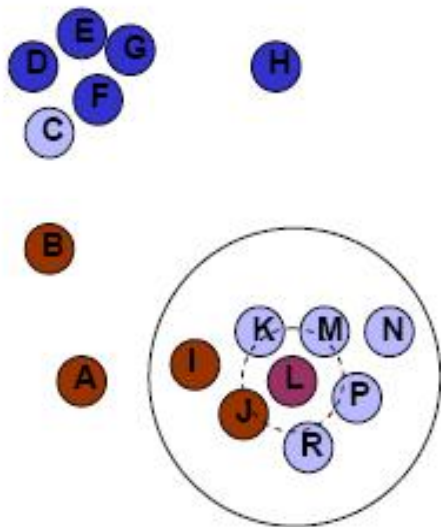
- Example Database (2-dimensional, 16 points)
- $\epsilon = 44$, $MinPts = 3$



seedlist: (L, 19) (K, 20) (R, 21) (M, 30) (P, 31) (C, 40)

OPTICS Algorithm

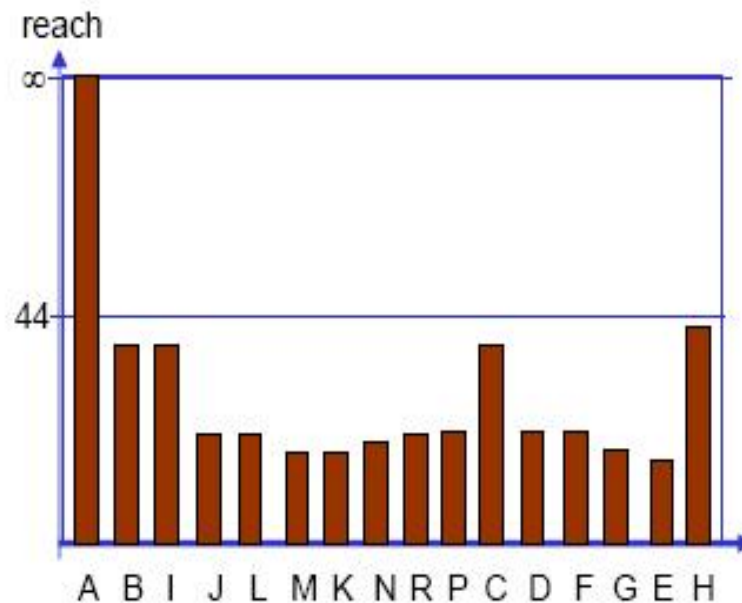
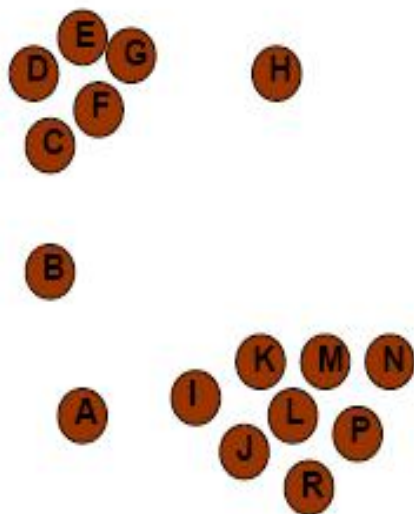
- Example Database (2-dimensional, 16 points)
- $\epsilon = 44$, $MinPts = 3$



seedlist: (M, 18) (K, 18) (R, 20) (P, 21) (N, 35) (C, 40)

OPTICS Algorithm

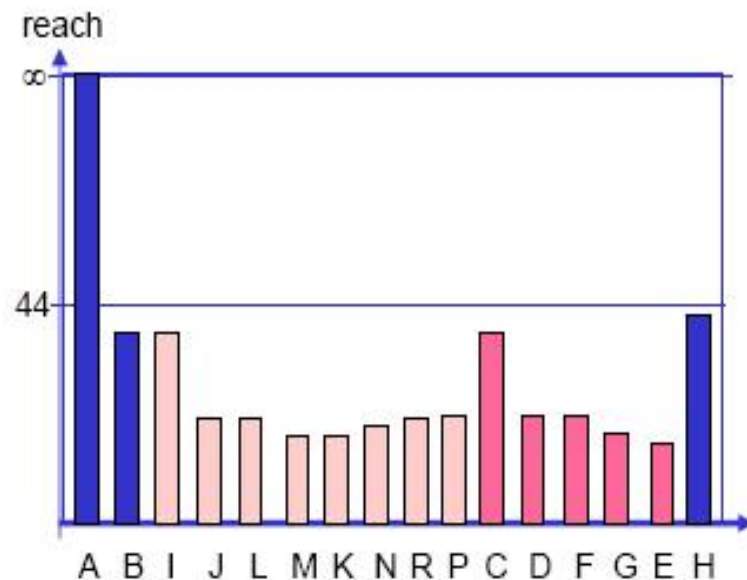
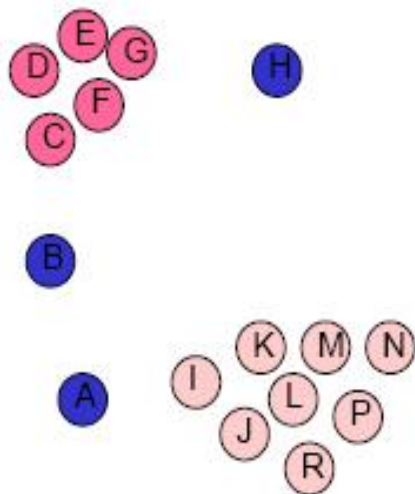
- Example Database (2-dimensional, 16 points)
- $\epsilon = 44$, $MinPts = 3$



seedlist: -

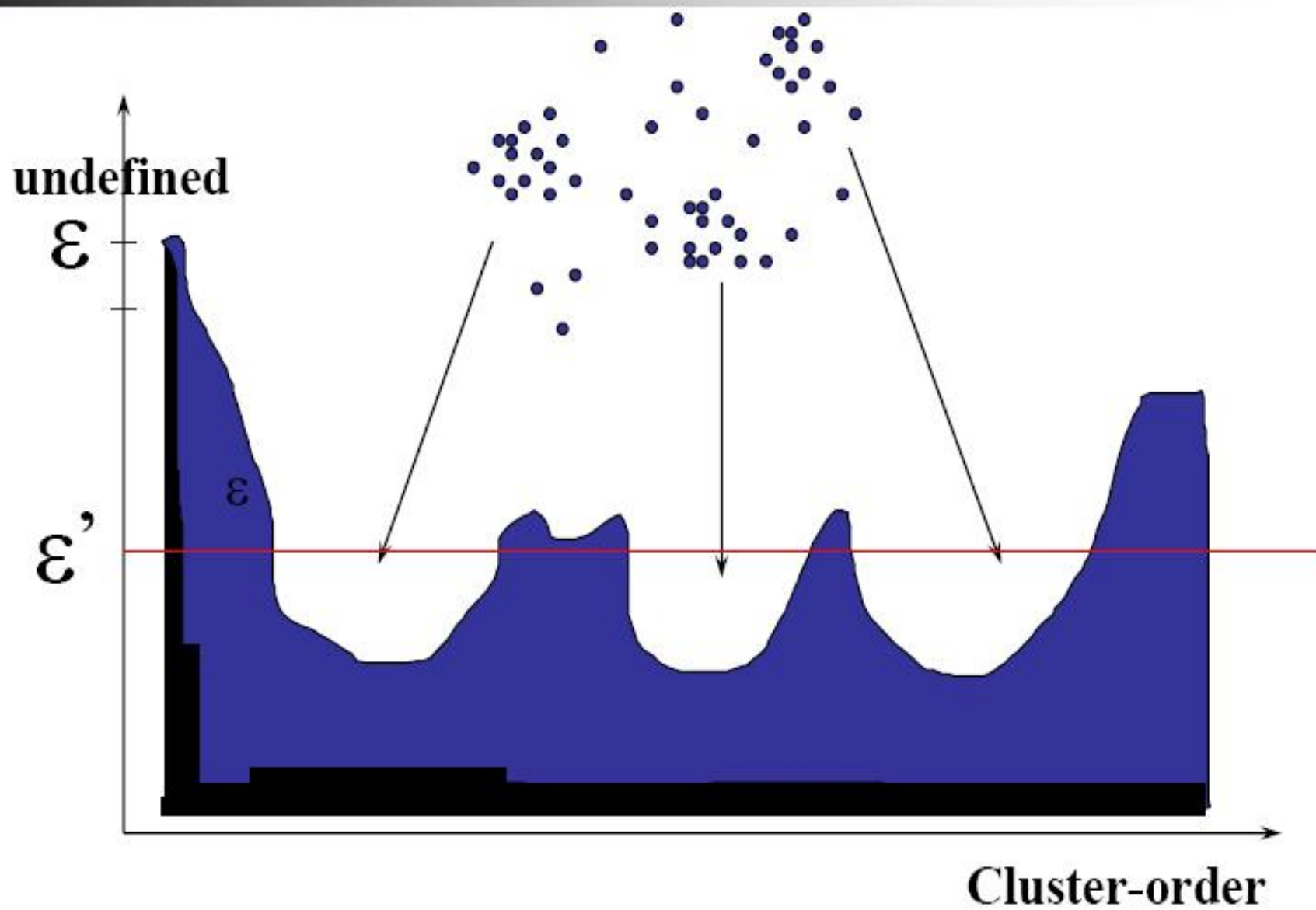
OPTICS Algorithm

- Example Database (2-dimensional, 16 points)
- $\epsilon = 44$, $MinPts = 3$



seedlist: -

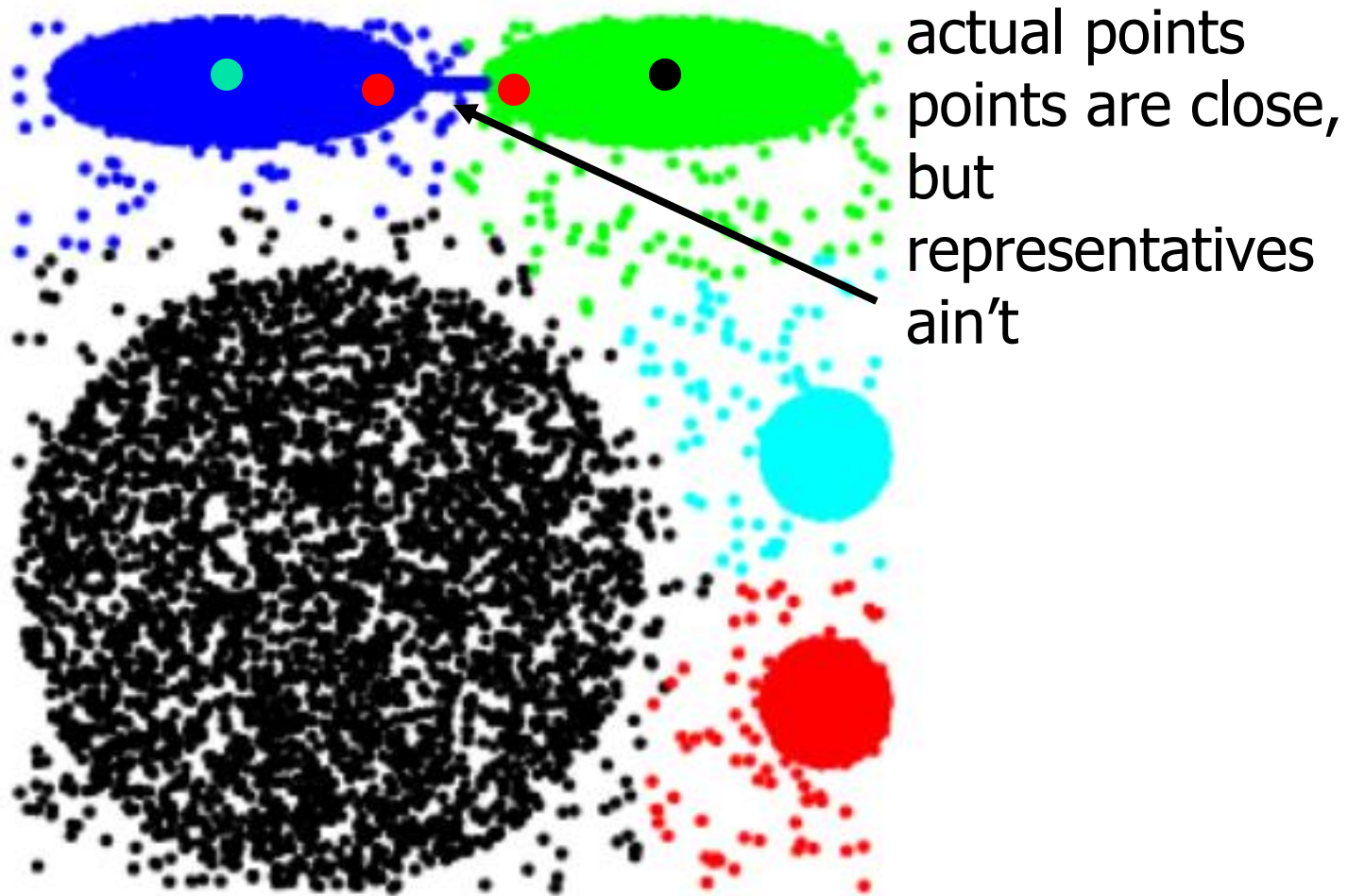
Reachability-distance



CURE

- instead of using all points in calculating the distance between a point and a cluster like DBSCAN, CURE uses a set of representative points within a cluster
- this could reduce the chaining effect

CURE diagram: no longer chains



聚类的评价指标

- **有聚类结果参考标准：外部指标 (external index)**
- **无聚类结果参考标准：内部指标 (internal index)**

聚类的评价指标

- 外部指标

- 假定存在某个参考聚类结果，通过与它比较计算得到外部指标

- 数据集： $D = \{x_1, x_2, \dots, x_m\}$

- 通过聚类给出的簇划分： $C = \{C_1, C_2, \dots, C_k\}$

- 参考聚类结果为： $C^* = \{C_1^*, C_2^*, \dots, C_s^*\}$

- a: 在 C 中和 C^* 中**都属于相同**聚类的样本对个数

- b: 在 C 中**属于相同**聚类、但在 C^* 中**属于不同**聚类的样本对个数

- c: 在 C 中**属于不同**聚类、但在 C^* 中**属于相同**聚类的样本对个数

- d: 在 C 中和 C^* 中**都属于不同**聚类的样本对个数

聚类的评价指标

● 外部指标

- a: 在C中和C*中**都属于相同**聚类的样本对个数
- b: 在C中**属于相同**聚类、但在C*中**属于不同**聚类的样本对个数
- c: 在C中**属于不同**聚类、但在C*中**属于相同**聚类的样本对个数
- d: 在C中和C*中**都属于不同**聚类的样本对个数

$$JC = \frac{a}{a + b + c}$$

Jaccard系数
(Jaccard Coefficient)

$$FMI = \sqrt{\frac{a}{a + b} \cdot \frac{a}{a + c}}$$

FM指数
(Fowlkes and Mallows Index)

$$RI = \frac{2(a + d)}{m(m - 1)}$$

Rand指数 (Rand Index)

聚类的评价指标

- 内部指标

—在无参考聚类结果下，根据**聚类簇样本之间的相互距离**定义一些内部评价指标

$$avg(C) = \frac{1}{|C||C-1|} \sum_{x_i, x_j \in C, x_i \neq x_j} dist(x_i, x_j)$$

簇内平均距离

$$max(C) = \max_{x_i, x_j \in C, x_i \neq x_j} dist(x_i, x_j)$$

簇内最远距离

$$d_{\min}(C_i, C_j) = \min_{x_i \in C_i, x_j \in C_j} dist(x_i, x_j)$$

簇间最近距离

$$d_{cen}(C_i, C_j) = dist(u_i, u_j), \quad u_i = \frac{1}{|C_i|} \sum_{x_j \in C_i} x_j$$

簇间中心距离

—DB指数 (Davies-Bouldin Index, DBI)

$$DBI = \frac{1}{k} \sum_{i=1}^k \max_{j \neq i} \left(\frac{avg(C_i) + avg(C_j)}{d_{cen}(C_i, C_j)} \right)$$

Note



聚类的“好坏”不存在绝对标准

**the goodness of clustering depends on
the opinion of the user**

故事一则

聚类的故事：

老师拿来苹果和梨，让小朋友分成两份。

小明把大苹果大梨放一起，小个头的放一起，老师点头，恩，体量感。

小芳把红苹果挑出来，剩下的放一起，老师点头，颜色感。

小武的结果？不明白。小武掏出眼镜：最新款，能看到水果里有几个籽，左边这堆单数，右边双数。

老师很高兴：新的聚类算法诞生了

聚类也许是机器学习中“新算法”出现最多、最快的领域
总能找到一个新的“标准”，使以往算法对它无能为力

挑战性问题

— 能够处理高维数据

- 既可处理属性较少的数据，也可处理属性较多的数据。
- 在高维空间聚类更具挑战性，随着维数的增加，具有相同距离的两个样本其相似程度可以相差很远。对于高维稀疏数据，这一点更突出。

— 对噪声鲁棒

- 在实际中，绝大多数样本集都包含噪声、空缺、部分未知属性、孤立点、甚至错误数据。

— 具有约束的聚类

- 在实际应用中，通常需要在某种约束条件下进行聚类，既满足约束条件，以希望有高聚类精度，是一个挑战性问题。

— 对初始输入参数鲁棒

- 具有自适应的簇数判定能力（一直没有解决好）。
- 对初始聚类中心鲁棒。

— 能够解决用户的问题

- 聚类结果能被用户所理解，并能带来经济效益，特别是在数据挖掘领域。